

Project Report

Script-Controlled ACL – Restrict Record Access Based on Field Value

Team ID	SWTID-2026-1268
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value

04 October 2026

Contents

1. Introduction
2. Ideation Phase
3. Requirement Analysis
4. Project Design
5. Project Planning & Scheduling
6. Functional and Performance Testing
7. Results
8. Advantages & Disadvantages
9. Conclusion
10. Future Scope
11. Appendix

1. Introduction

1.1 Project Overview

In many systems, anyone who has access to a table can open every record in it, even when some records are meant for one department or group only. This project restricts record access based on the value of a field. An Access Control List (ACL) script on the ServiceNow platform reads the controlling field of a record (for example, Department) and compares it with the logged-in user's roles and attributes. The record is shown only when the script returns true; otherwise access is denied.

1.2 Purpose

The purpose of the project is to protect sensitive records by making access depend on data in the record itself, not only on table-level roles. It aims to:

- show each user only the records they are permitted to see, in lists, forms, reports and API calls;
- deny access by default and fail safe when a script error or empty field occurs;
- let administrators change access rules without rebuilding the application;
- keep an audit log of allowed and denied attempts.

2. Ideation Phase

2.1 Problem Statement

Problem: Administrators and data owners need a way to keep records visible only to the right people, but table-level roles are too broad. Staff can open records that belong to other departments, which risks leaks of sensitive information.

Need: A rule that decides access per record, using a field value in the record, that is easy to maintain and cannot be bypassed through lists, URLs or the API.

2.2 Empathy Map Canvas

Empathy map for the administrator / data owner:

Says "Only HR should see HR records." "How do I stop people opening records by URL?"	Thinks Table roles are too broad. One mistake could expose private data.
Does Creates roles and ACLs, tests with different users, checks logs.	Feels Worried about data leaks; wants confidence and control.

Pains: broad table access, manual checks, hard-to-trace access. **Gains:** record-level protection, clear deny messages, an audit trail.

2.3 Brainstorming

Ideas considered for restricting record access:

- Role-only ACLs on the table (simple but too broad).
- Field-value conditions in the ACL (no code, limited logic).
- Script-controlled ACL using the record field value and user attributes (flexible).
- Separate tables per department (hard to maintain and report on).
- Data masking of sensitive fields only.

Selected idea: a script-controlled ACL combined with field-value conditions and field masking, because it is flexible, reusable through script includes, and works for UI, list and API access.

3. Requirement Analysis

3.1 Customer Journey Map

Stage	User action	Touchpoint	Feeling	Opportunity
Login	Signs in with email/password or SSO	Login page	Confident	Fast SSO
Open records	Opens a list or a record	List / form	Curious	Show only permitted records
Access check	ACL script compares field value and roles	ACL engine	Unaware of the check	Quick decision
Result	Sees the record or an access denied message	Form / message	Satisfied or informed	Clear message, no data leak
Review	Auditor reads the access log	Audit report	In control	Easy export

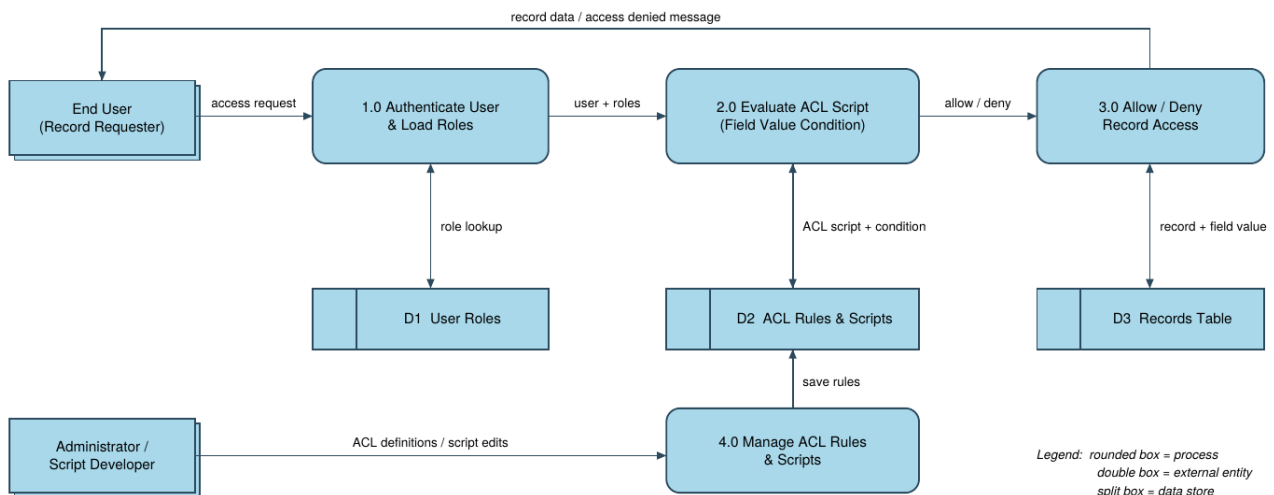
3.2 Solution Requirement

Functional requirements:

FR No.	Functional Requirement	Sub Requirement
FR-1	User Authentication & Roles	Login with email/password or SSO; role assignment
FR-2	ACL Rule Configuration	Create ACL on table and field; define field-value condition; enable/disable
FR-3	Script-Controlled Access	Write and test ACL script (true/false); reuse via script includes
FR-4	Record Access Enforcement	Filter lists; block direct URL/API access; mask fields; access denied message
FR-5	Audit & Reporting	Log allowed/denied attempts; export report

Non-functional requirements: usability, security (deny by default, server-side enforcement), reliability (fail safe), performance (under about 200 ms added per request), availability (target 99.9%) and scalability.

3.3 Data Flow Diagram



The end user's request is authenticated (1.0), the ACL script is evaluated using the record field value (2.0), and access is allowed or denied (3.0). Administrators manage rules and scripts through process 4.0. Data stores are User Roles (D1), ACL Rules & Scripts (D2) and the Records Table (D3).

3.4 Technology Stack

Component	Technology
User Interface	ServiceNow Next Experience UI / Service Portal (HTML, CSS, JavaScript, AngularJS)
Application logic	ACL rules; server-side JavaScript ACL scripts; Script Includes
Database	ServiceNow relational database (Now Platform tables)
File storage	ServiceNow attachment storage
External interfaces	SAML 2.0 SSO, LDAP; REST Table API
Machine learning	Not used (rule-based)
Infrastructure	ServiceNow cloud instance (Personal Developer Instance for development)

4. Project Design

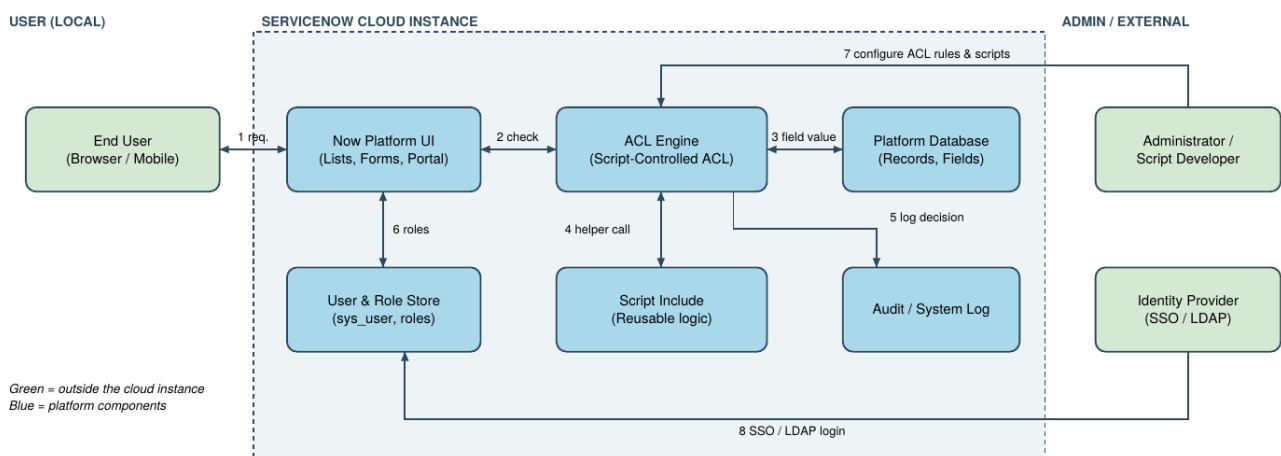
4.1 Problem Solution Fit

Problem	Solution fit
Table roles give access to every record	ACL checks the field value of each record
Restricted records can be opened by URL or API	ACL is enforced on the server for forms, lists, reports and API calls
Rules are hard to change	Rules and scripts are edited by administrators and shared through script includes
No visibility into who tried to access what	Audit log of allowed and denied attempts

4.2 Proposed Solution

An ACL is created on the table with the controlling field. Its script reads the record's field value and the current user's roles and returns true or false. Access is denied by default, script errors deny access, sensitive fields can be masked, and each decision is logged. Administrators can enable or disable rules without deleting them, and developers can test scripts on sample records before activation.

4.3 Solution Architecture



The user's browser or mobile device talks to the Now Platform UI. The ACL engine checks the request, reads the record's field value from the platform database, looks up roles, calls reusable script includes, and writes the decision to the audit log. Administrators configure rules, and SSO/LDAP provides login.

5. Project Planning & Scheduling

5.1 Project Planning

The work is split into four sprints using story points of 1, 2, 3 and 5.

Sprint	Epics and stories	Story points
Sprint 1	Authentication & Roles (USN-1, 2, 9); ACL Configuration (USN-3, 4)	11
Sprint 2	ACL Configuration (USN-5); Script-Controlled ACL (USN-6, 7); Record Access (USN-10, 11)	17
Sprint 3	Script-Controlled ACL (USN-8); Record Access (USN-12); Audit & Reporting (USN-14)	11
Sprint 4	Record Access (USN-13); Audit & Reporting (USN-15)	5
Total		44

Velocity = 44 story points / 4 sprints = **11 story points per sprint**.

6. Functional and Performance Testing

6.1 Performance Testing

Test	What is tested	Expected result
FT-01	Text input validation in ACL form fields	Valid inputs accepted; errors for invalid inputs
FT-02	Field-value condition validation	Matching value grants access; non-matching or empty denies
FT-03	ACL enforcement in lists, forms and masked fields	Permitted users see records; others are denied
FT-04	REST Table API with valid and invalid users	Permitted users succeed; others are rejected
PT-01	Response time with the ACL script active	Under 3 seconds
PT-02	Concurrent API calls	No slowdown
PT-03	Large list load with the ACL script running	Works without crashing

User Acceptance Testing covered 13 scenarios, including positive and negative cases, direct URL access, script errors, field masking, API access and the audit log. See the separate UAT and Functional & Performance Testing documents for full results.

7. Results

7.1 Output Screenshots

Add screenshots of the following outputs:

[Insert screenshot: ACL record showing the table and controlling field]

[Insert screenshot: ACL script]

[Insert screenshot: HR user opening a permitted record]

[Insert screenshot: Finance user seeing the access denied message]

[Insert screenshot: List view showing only permitted records]

[Insert screenshot: Masked field on a record]

[Insert screenshot: Access log report]

8. Advantages & Disadvantages

Advantages

- Record-level protection based on real data in the record.
- Works across forms, lists, reports and API calls.
- Rules can be changed or disabled without rebuilding the application.
- Reusable script logic and a clear audit trail.

Disadvantages

- Scripts need careful writing and testing; mistakes can block or expose records.
- Heavy queries inside ACL scripts can slow down large lists.
- Depends on correct and complete field data (empty values are denied).
- Maintaining many scripts can become complex over time.

9. Conclusion

The project shows that a script-controlled ACL can limit access to each record based on a field value and the user's roles. With deny-by-default behaviour, field masking and an audit log, it gives administrators finer control over sensitive data than table-level roles alone.

10. Future Scope

- Admin screen to build field-value rules without writing code.
- Alerts or dashboards for repeated denied attempts.
- Policy templates for common cases such as department, region or ownership.
- Automatic performance checks for slow ACL scripts.
- Review workflow for approving ACL script changes.

11. Appendix

Source Code (sample ACL script)

```
// ACL script (illustrative - adjust table and field names)
(function() {
  var userDept = gs.getUser().getRecord().getValue('department');
  var recordDept = current.getValue('u_department');

  // Deny by default; allow only when departments match
  if (!recordDept) { return false; } // empty field -> deny
  answer = (recordDept == userDept) || gs.hasRole('admin');
})();
```

Dataset Link

Not applicable (no external dataset is used; test records are created in the instance).

GitHub & Project Demo Link

GitHub: *[add link]*

Demo video: *[add link]*